

CMPT 280

Tutorial: Specifications

Stacks: A Refresher

Refresher

- A "last-in, first out" (LIFO) container
- Three fundamental operations:
 - Put new item on top of stack
 - Look at item at top of stack
 - Remove the item on the top of the stack.
- Stacks are also a type of *dispenser* (defined in Chapter 10 of the textbook).

Stack Specifications

What does a stack actually need?

Methods

- newStack
- Push
- Pop
- Top
- isEmpty
- isFull

Sets

- set of all stacks
- set of items that can be in a stack
- booleans: {**true**, **false**}

Stack Specifications

Name: Stack< G >

Sets:

S : set of all stacks containing elements from G

G : set of items that can be in the stack S

B : {true, false}

Signatures:

newStack< G >: $\rightarrow S$

S .isEmpty: $\rightarrow B$

S .isFull: $\rightarrow B$

S .push(g): $G \nrightarrow S$

S .pop: $\nrightarrow S$

S .top: $\nrightarrow G$

Preconditions: $\forall s \in S, g \in G$

newStack< G > : none

s .isEmpty : none

s .isFull : none

s .push : s is not full

s .pop : s is not empty

s .top : s is not empty

Semantics: $\forall s \in S, g \in G$

newStack< G > : Construct a new stack that can store elements of G

s .isEmpty: return true if s is empty, false otherwise

s .isFull: return true if s is full, false otherwise

s .push(g): push g onto top of stack s .pop: remove top element g from stack s .top: fetch top element g from stack

Stack Specifications

Name: Stack< G >

Sets:

S : set of all stacks containing elements from G

G : set of items that can be in the stack S

B : {true, false}

Signatures:

newStack< G >: $\rightarrow S$

S .isEmpty: $\rightarrow B$

S .isFull: $\rightarrow B$

S .push(g): $G \nrightarrow S$

S .pop: $\nrightarrow S$

S .top: $\nrightarrow G$

Preconditions: $\forall s \in S, g \in G$

newStack< G > : none

s .isEmpty : none

s .isFull : none

s .push : s is not full

s .pop : s is not empty

s .top : s is not empty

Semantics: $\forall s \in S, g \in G$

newStack< G > : Construct a new stack that can store elements of G

s .isEmpty: return true if s is empty, false otherwise

s .isFull: return true if s is full, false otherwise

s .push(g): push g onto top of stack s .pop: remove top element g from stack s .top: fetch top element g from stack

Stack Specifications

Name: Stack< G >

Sets:

S : set of all stacks containing elements from G

G : set of items that can be in the stack S

B : {true, false}

Signatures:

newStack< G >: $\rightarrow S$

S .isEmpty: $\rightarrow B$

S .isFull: $\rightarrow B$

S .push(g): $G \nrightarrow S$

S .pop: $\nrightarrow S$

S .top: $\nrightarrow G$

Preconditions: $\forall s \in S, g \in G$

newStack< G > : none

s .isEmpty : none

s .isFull : none

s .push : s is not full

s .pop : s is not empty

s .top : s is not empty

Semantics: $\forall s \in S, g \in G$

newStack< G > : Construct a new stack that can store elements of G

s .isEmpty: return true if s is empty, false otherwise

s .isFull: return true if s is full, false otherwise

s .push(g): push g onto top of stack s .pop: remove top element g from stack s .top: fetch top element g from stack

Stack Specifications

Name: Stack< G >

Sets:

S : set of all stacks containing elements from G

G : set of items that can be in the stack S

B : {true, false}

Signatures:

newStack< G >: $\rightarrow S$

S .isEmpty: $\rightarrow B$

S .isFull: $\rightarrow B$

S .push(g): $G \nrightarrow S$

S .pop: $\nrightarrow S$

S .top: $\nrightarrow G$

Preconditions: $\forall s \in S, g \in G$

newStack< G > : none

s .isEmpty : none

s .isFull : none

s .push : s is not full

s .pop : s is not empty

s .top : s is not empty

Semantics: $\forall s \in S, g \in G$

newStack< G > : Construct a new stack that can store elements of G

s .isEmpty: return **true** if s is empty, **false** otherwise

s .isFull: return **true** if s is full, **false** otherwise

s .push(g): push g onto top of stack s .pop: remove top element g from stack s .top: fetch top element g from stack

Implementation

- Specifications done in this way can be translated into any language.
- We only happen to be using Java.

Implementation of Signatures

Signatures translate to method headers.

- $\text{newStack}\langle G \rangle \rightarrow S$
- $S.\text{isEmpty} \rightarrow B$
- $S.\text{isFull} \rightarrow B$
- $S.\text{push}(g) \quad G \nrightarrow S$
- $S.\text{pop} \nrightarrow S$
- $S.\text{top} \nrightarrow G$

$\Rightarrow$

```
1 public class Stack<G> {  
2     public Stack() {}  
3     public boolean isEmpty() {}  
4     public boolean isFull() {}  
5     public void push(G g) {}  
6     public void pop() {}  
7     public G top() {}  
8 }  
9  
10  
11  
12  
13  
14  
15
```

Implementation of Signatures

Signatures translate to method headers.

- $\text{newStack}\langle G \rangle \rightarrow S$
- $S.\text{isEmpty} \rightarrow B$
- $S.\text{isFull} \rightarrow B$
- $S.\text{push}(g) \quad G \nrightarrow S$
- $S.\text{pop} \nrightarrow S$
- $S.\text{top} \nrightarrow G$

$\Rightarrow$

```
1 public class Stack<G> {  
2     public Stack() {}  
3     public boolean isEmpty() {}  
4     public boolean isFull() {}  
5     public void push(G g) {}  
6     public void pop() {}  
7     public G top() {}  
8 }  
9  
10  
11  
12  
13  
14  
15
```

Implementation of Preconditions

Preconditions are javadocs, exceptions, and if-statements.

- `newStack<G>` : none
- `s.isEmpty` : none
- `s.isFull` : none
- `s.push` : `s` is not full
- `s.pop` : `s` is not empty
- `s.top` : `s` is not empty

⇒

```
1  /** @precond the stack is not full */
2  public void push(G g)
3      throws IllegalStateException {
4      if (this.isFull()) {
5          throw new
6              IllegalStateException();
7      }
8
9  /** @precond the stack is not empty */
10 public void pop()
11     throws IllegalStateException {
12     if (this.isEmpty()) {
13         throw new
14             IllegalStateException();
15     }
16
17 /** @precond the stack is not empty */
18 public G top()
19     throws IllegalStateException {
20     if (this.isEmpty()) {
21         throw new
22             IllegalStateException();
23     }
24 }
```

Implementation of Preconditions

Preconditions are javadocs, exceptions, and if-statements.

- `newStack<G>` : none
- `s.isEmpty` : none
- `s.isFull` : none
- `s.push` : `s` is not full
- `s.pop` : `s` is not empty
- `s.top` : `s` is not empty

⇒

```
1  /** @precond the stack is not full */
2  public void push(G g)
3      throws IllegalStateException {
4      if (this.isFull()) {
5          throw new
6              IllegalStateException();
7      }
8
9  /** @precond the stack is not empty */
10 public void pop()
11     throws IllegalStateException {
12     if (this.isEmpty()) {
13         throw new
14             IllegalStateException();
15     }
16
17 /** @precond the stack is not empty */
18 public G top()
19     throws IllegalStateException {
20     if (this.isEmpty()) {
21         throw new
22             IllegalStateException();
23     }
24 }
```

Implementation of Semantics

Semantics become the javadoc comments (and later, code).

- `newStack<G>` : Construct a new stack to hold $g \in G$
- `s.isEmpty`: return **true** if `s` is empty, **false** otherwise
- `s.isFull`: return **true** if `s` is full, **false** otherwise

⇒

```
1  public class Stack<G> {
2
3      /**
4       * Create a new Stack
5       */
6      public Stack() {}
7
8      /**
9       * Tests whether the stack is empty
10     * @returns true if the stack is empty,
11     *           false otherwise
12     */
13     public boolean isEmpty() {}
14
15     /**
16     * Tests whether the stack is full
17     * @returns true if the stack is full,
18     *           false otherwise
19     */
20     public boolean isFull() {}
21
22     [...]
```

Implementation of Semantics

Semantics become the javadoc comments (and later, code).

- `newStack<G>` : Construct a new stack to hold $g \in G$
- `s.isEmpty`: return **true** if `s` is empty, **false** otherwise
- `s.isFull`: return **true** if `s` is full, **false** otherwise

⇒

```
1  public class Stack<G> {
2
3      /**
4       * Create a new Stack
5       */
6      public Stack() {}
7
8      /**
9       * Tests whether the stack is empty
10      * @returns true if the stack is empty,
11      *           false otherwise
12      */
13     public boolean isEmpty() {}
14
15     /**
16      * Tests whether the stack is full
17      * @returns true if the stack is full,
18      *           false otherwise
19      */
20     public boolean isFull() {}
21
22     [...]
```

Implementation of Semantics

- $s.\text{push}(g)$: push g onto top of stack
- $s.\text{pop}$: remove top element g from stack

⇒

```
1  [...]
2
3  /**
4   * Pushes element g onto
5   *   the top of the stack
6   * @precond the stack is not full
7   */
8  public void push(G g)
9      throws IllegalStateException {
10     if (this.isFull()) {
11         throw new
12             IllegalStateException();
13     }
14
15     /**
16      * Removes the top element
17      *   from the stack
18      * @precond the stack is not empty
19      */
20     public void pop()
21         throws IllegalStateException {
22         if (this.isEmpty()) {
23             throw new
24                 IllegalStateException();
25         }
26
27     [...]
```

Implementation of Semantics

- $s.\text{push}(g)$: push g onto top of stack
- $s.\text{pop}$: remove top element g from stack

⇒

```
1  [...]
2
3  /**
4   * Pushes element g onto
5   *   the top of the stack
6   * @precond the stack is not full
7   */
8  public void push(G g)
9      throws IllegalStateException {
10     if (this.isFull()) {
11         throw new
12             IllegalStateException();
13     }
14
15     /**
16      * Removes the top element
17      *   from the stack
18      * @precond the stack is not empty
19      */
20     public void pop()
21         throws IllegalStateException {
22         if (this.isEmpty()) {
23             throw new
24                 IllegalStateException();
25         }
26
27     [...]
```


Implementation of Semantics

- $s.top$: fetch top element g from stack

$\Rightarrow$

```
1  [...]
2
3  /**
4   * Returns the top element
5   *   from the stack
6   * @precond the stack is not empty
7   * @return the top element of the stack
8   */
9  public G top()
10     throws IllegalStateException {
11     if (this.isEmpty()) {
12         throw new
13             IllegalStateException();
14     }
15
16
17 }
```

Implementation of Semantics

- $s.top$: fetch top element g from stack

$\Rightarrow$

```
1  [...]
2
3  /**
4   * Returns the top element
5   *   from the stack
6   * @precond the stack is not empty
7   * @return the top element of the stack
8   */
9  public G top()
10     throws IllegalStateException {
11     if (this.isEmpty()) {
12         throw new
13             IllegalStateException();
14     }
15
16
17 }
```

Implementation - the rest

- Add the data structures and method implementations
- Don't forget to actually check your preconditions
- ADTs can be implemented in any language, using any implementation